

Figure 6: Analysing data

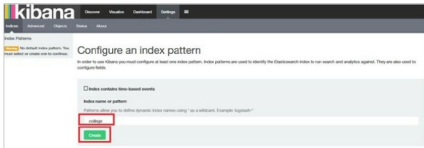


Figure 7: Configuring Kibana



Figure 8: Different visualisation options

To search the data, use `GET /college/students/_search`, which will return all students' records.

To search for data, where the gender is male, use: `GET /college/students/_search?q=gender:Male`

...and it will return all male students as shown in Figure 5.

In the results, we can see that of a total of five student documents, three

are of males.

Similarly, we can search the data using other fields. But this syntax will be difficult for *and/or* criteria. Elasticsearch provides query DSL or the Domain Specific Language to get the data. Learning DSL will take some time. The following sections show how to write queries using DSL. For example, to search all male students

who also have the hobby of reading, use the commands given below:

```
GET /college/students/_search
{
  "query":{
    "bool": {
      "must" :{"match":{"hobbies":
"reading"}},
      "must"
:{"match":{"gender":"Male"}}
    }
  }
}
```

Let's search the goal field to find what contains the text 'works as', as follows:

```
GET /college/students/_search
{
  "query":{
    "match" : {
      "goal" : "work as"
    }
  }
}
```

This will display all the students whose *goal* contains the words 'works as' or 'work'.

We have covered how to store and search for data using Elasticsearch. Now, the last part is to analyse the data.

Analysing data: To analyse the data, aggregators are provided something like 'group by' in SQL. For example, if we need to find the number of male and female students, refer to Figure 6.

```
GET /college/students/_search
{
  "aggs":{
    "Gender" : {
      "terms" : {"field" : "gender"}
    }
  }
}
```

There are many other ways to think of for aggregators. For more details, you could refer to online documentation.